

AST Generation

10 câu harmony.

232

Cho đoạn ngữ pháp được viết trong ANTLR4 cho ngôn ngữ BMirror như sau:

```
program: assign_stmt* EOF;
assign_stmt: assign NEW_LINE;
assign: ID CM assign CM exp | ID EQ exp;
exp: exp (PLUS | MINUS) term | term;
term: term (MUL | DIV) fact | fact;
fact: ID | INTLIT | idx_op;
idx_op: ID (LB INTLIT RB)+;
ID: [a-z]+; CM: ','; SM: ';'; EQ: '=';
INTLIT: [0-9]+; LB: '['; RB: ']'; NEW_LINE: '\r'? '\n';
PLUS: '+'; MINUS: '-'; MUL: '*'; DIV: '/';
WS: [ \t] → skip;
```

và các lớp AST được khai báo trong ngôn ngữ Python3 như sau:

```
class AST
class Program(AST): # stmts: List[Stmt]
class Stmt(AST)
class Assign(Stmt): # lhs: Id, right: Exp
class Exp(AST)
class BinExp(Exp): # op: str, left: Exp, right: Exp
class IdxOp(Exp): # base: Exp, idx: int
class IntLit(Exp): # value: int
class Id(Exp): # name: str
```

Với chuỗi nhập viết trên ngôn ngữ BMirror như sau:

```
a, b, c = 1 + 2, a * 2, 4 / b
d = arr[1][2][3]
```

Đoạn mã sau với một số chỗ trống và một số đoạn bị che (## Hidden code) được sử dụng để thực hiện việc sinh AST từ câu cú pháp được tạo ra bởi bộ phân tích cú pháp do ANTLR sinh ra trên Python3 là:

```

class ASTGenerator(BMirrorVisitor):
    def visitProgram(self, ctx):
        assign_stmts = []
        for assign in ctx.assign_stmt(): assign_stmts += self.visit(assign)
        return Program(assign_stmts)
    def visitAssign_stmt(self, ctx):
        ids, explist = self.visit(ctx.assign())
        return ____ (1) ____
    def visitAssign(self, ctx):
        if ctx.getChildCount() == 3:
            return ____ (2) ____
        ids, explist = self.visit(ctx.assign())
        return [Id(ctx.ID().getText())] + ____ (3) ____ + [self.visit(ctx.exp())]
    def visitExp(self, ctx):
        ## Hidden Code
    def visitTerm(self, ctx):
        ## Hidden Code
    def visitFact(self, ctx):
        if ctx.ID(): return Id(ctx.ID().getText())
        elif ctx.INTLIT(): return IntLit(int(ctx.INTLIT().getText()))
        return ____ (4) ____
    def visitIdx_op(self, ctx):
        return reduce(lambda x,y:____ (5) ____,____ (6) ____ ____ (7) ____ )

```

1. **(Câu 36 - 232)** Kiểu đầy đủ của tham số ctx trong phương thức `visitAssign_stmt` là

- a. `BMirrorParser.Assign_stmtContext`
- b. `BMirrorParserTree.Assign_stmtContext`
- c. `BMirror.Assign_stmtContext`
- d. `BMirrorParser.Assign_stmt`

2. **(Câu 37 - 232)** Chỗ trống (7) ở dòng số 23 nên là

- a. `,Id(ctx.ID().getText())`
- b. `,[]`
- c. `,ctx.ID()`
- d. Không cần mã cho chỗ trống này

3. **(Câu 38 - 232)** Chỗ trống (6) ở dòng số 23 nên là

- a. `ctx.ID()`
- b. `int(ctx.INTLIT().getText())`

c. `self.visit(ctx.INTLIT())`

d. `ctx.INTLIT()`

4. (Câu 39 - 232) Chỗ trống (5) ở dòng số 23 nên là

a. `IdxOp(y,x)`

b. `IdxOp(x,int(y))`

c. `IdxOp(x,int(y.getText()))`

d. `IdxOp(y,int(x))`

5. (Câu 40 - 232) Chỗ trống (4) ở dòng số 21 nên là

a. `[self.visit(ctx.idx_op())]`

b. `IdxOp(self.visit(ctx.idx_op()))`

c. `self.visit(ctx.idx_op())`

d. `ctx.idx_op()`

6. (Câu 41 - 232) Nhận định nào sau đây là đúng?

a. Số loại đối tượng mà `visitExp` và `visitFact` có thể trả về là giống nhau và là 4.

b. `visitFact` có thể trả về đối tượng `Id` trong khi `visitTerm` thì chỉ có thể trả về đối tượng `BinExp`

c. `visitFact` có thể trả về đối tượng `IntLit` trong khi `visitExp` thì chỉ có thể trả về đối tượng `Exp`

d. Số loại đối tượng mà `visitExp` và `visitTerm` có thể trả về là 1 và là `BinExp`.

7. (Câu 42 - 232) Chỗ trống (2) ở dòng số 11 nên là

a. `Assign(Id(ctx.ID().getText()), self.visit(ctx.exp()))`

b. `[Id(ctx.ID().getText())], [self.visit(ctx.exp())]`

c. `Assign(Id(ctx.ID().getText()), ctx.exp())`

d. `(Id(ctx.ID().getText()), self.visit(ctx.exp()))`

8. (Câu 43 - 232) Chỗ trống (3) ở dòng số 13 nên là

a. `self.visit(ctx.assign())`

b. `ids,explist`

c. `ids+explist`

d. `ctx.assign()`

9. (Câu 44 - 232) Biết rằng `zip` là hàm để tạo một danh sách các phần tử tuple được lấy từ các phần tử có chỉ số tương ứng của các danh sách đầu vào. Chỗ trống (1) ở dòng số 8 nên là

- a. `[Assign(idc, exp) for idc, exp in list(zip(ids[::-1], explist))]`
- b. `[Assign(idc, exp) for idc, exp in list(zip(ids[::-1], explist[::-1]))]`
- c. `[Assign(idc, exp) for idc, exp in list(zip(ids, explist[::-1]))]`
- d. `[Assign(idc, exp) for idc, exp in list(zip(ids, explist))]`

10. **(Câu 45 - 232)** Chọn lệnh phù hợp để thay thế các dòng lệnh từ 3 đến 5?

- a. `return Program(reduce(lambda prev, curr: prev + self.visit(curr), ctx.assign_stmt()))`
- b. `return Program(reduce(lambda prev, curr: prev + [self.visit(curr)], ctx.assign_stmt()))`
- c. `return Program(reduce(lambda prev, curr: prev + self.visit(curr), ctx.assign_stmt(), []))`
- d. `return Program(reduce(lambda prev, curr: prev + [self.visit(curr)], ctx.assign_stmt(), []))`